

P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-tuning Universally Across Scales and Tasks

자연어 처리팀:

조해창, 변현정, 허정원, 정천감, 박산희, 이기성

Contents

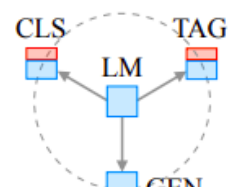
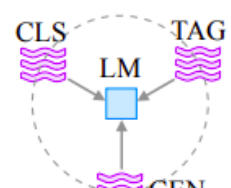
- Preliminaries
- P-tunning V2
- Code
- Another Use case

Preliminaries

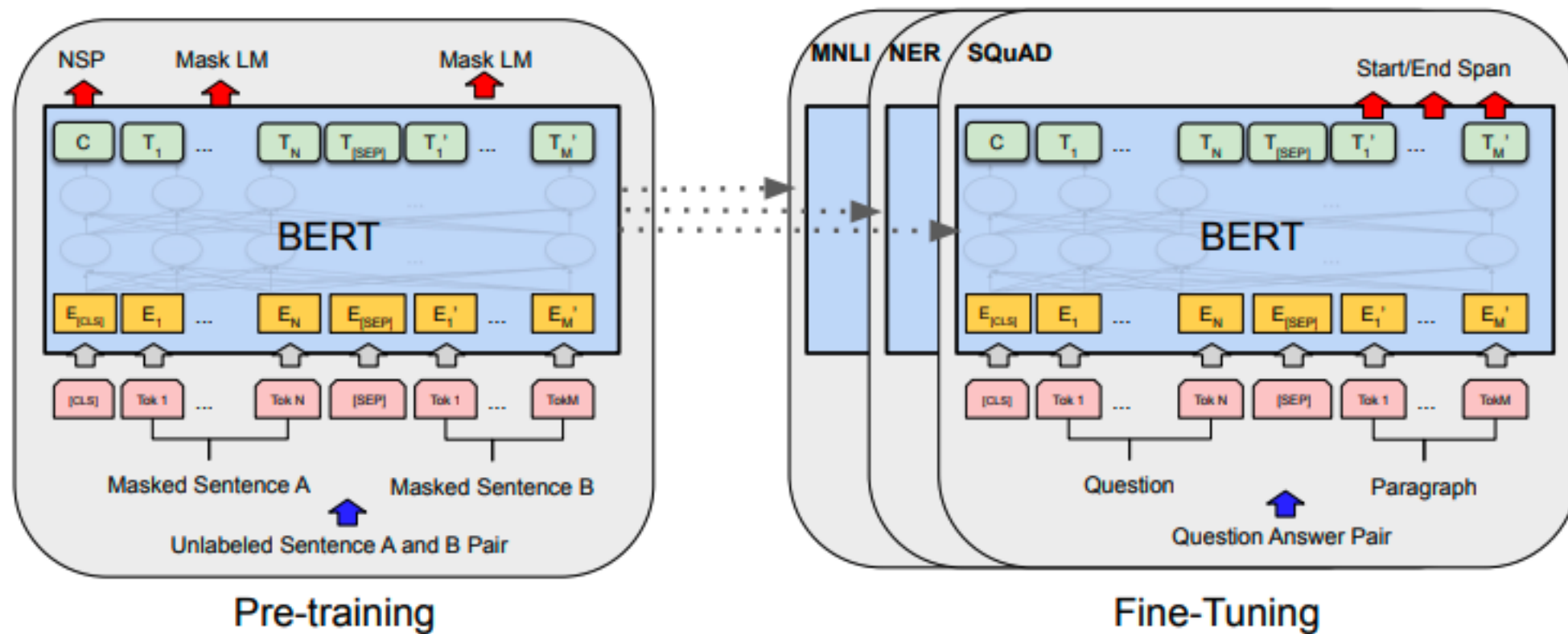
- Fine tuning
- Prompting
- Verbalizer
- Prompt tuning

Preliminaries

- Fine tuning
- Prompting
- Prompt tuning
- Verbalizer

Paradigm	Engineering	Task Relation
a. Fully Supervised Learning (Non-Neural Network)	Features (e.g. word identity, part-of-speech, sentence length)	
b. Fully Supervised Learning (Neural Network)	Architecture (e.g. convolutional, recurrent, self-attentional)	
c. Pre-train, Fine-tune	Objective (e.g. masked language modeling, next sentence prediction)	
d. Pre-train, Prompt, Predict	Prompt (e.g. cloze, prefix)	

Fine tuning



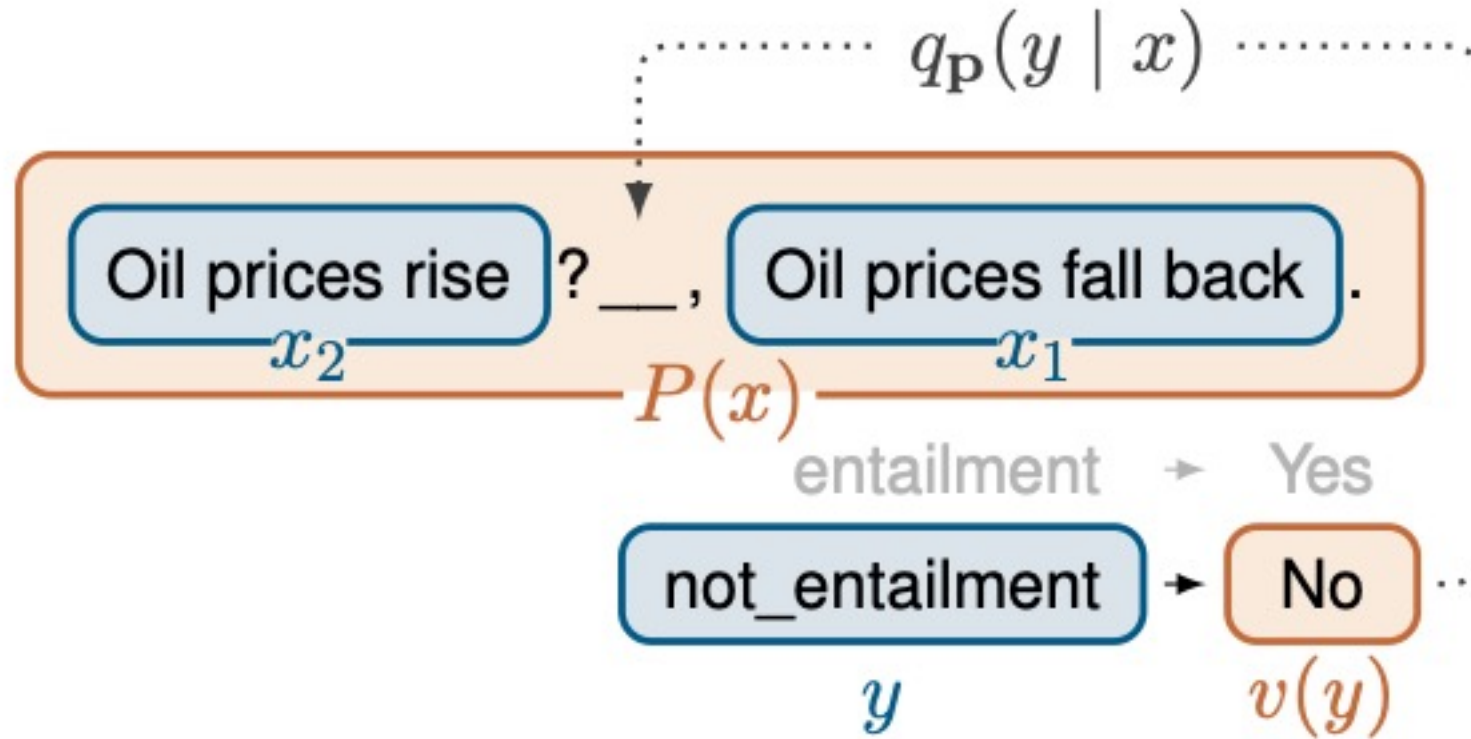
Prompting

[CLS] Best pizza ever!

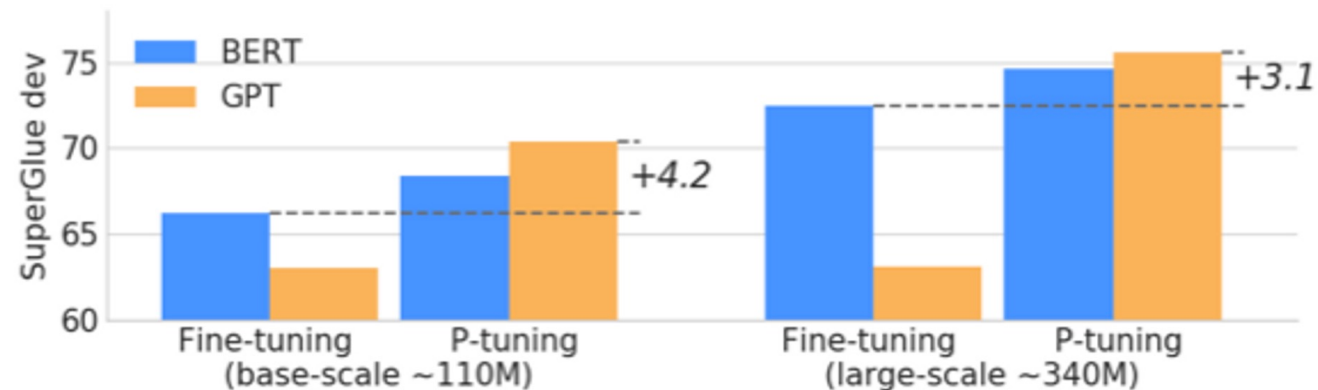
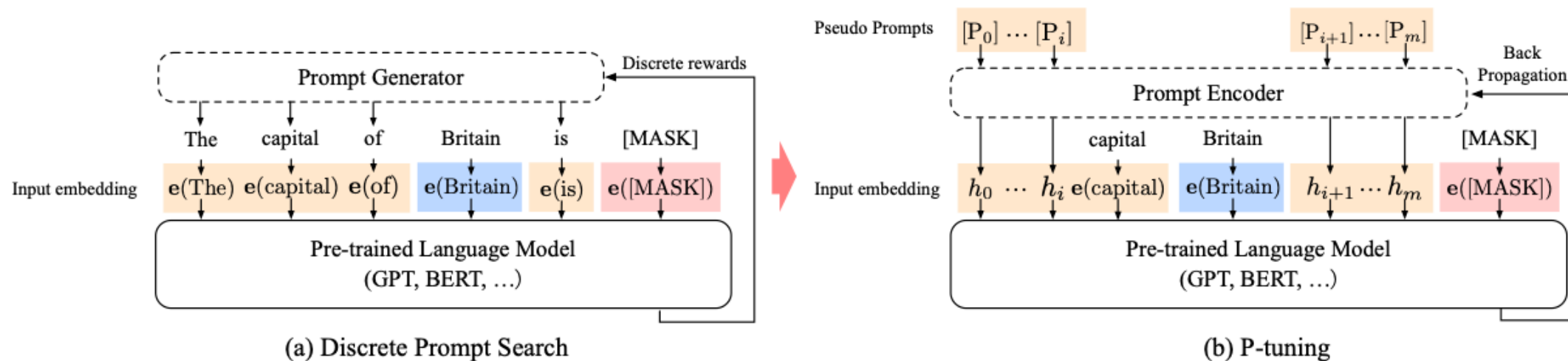


Best pizza ever! It was ____.

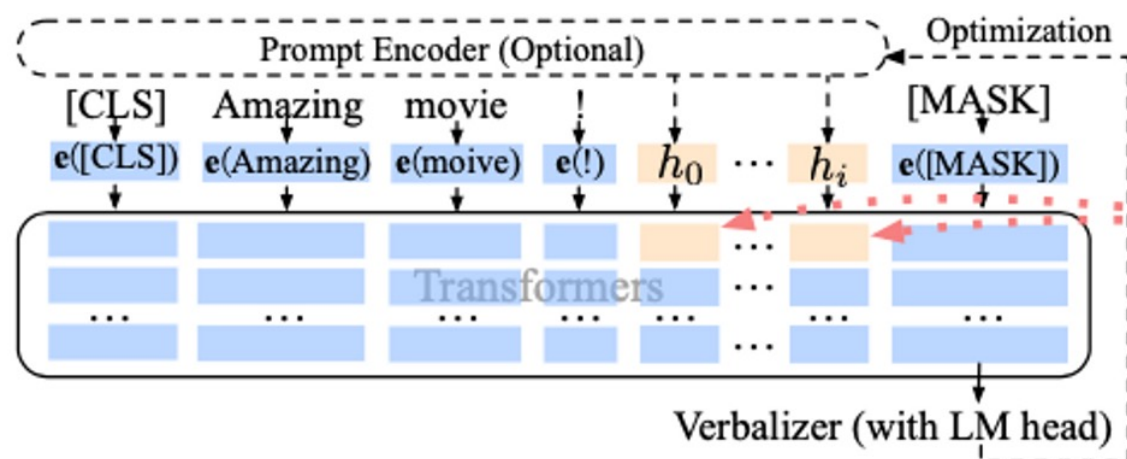
Verbalizer



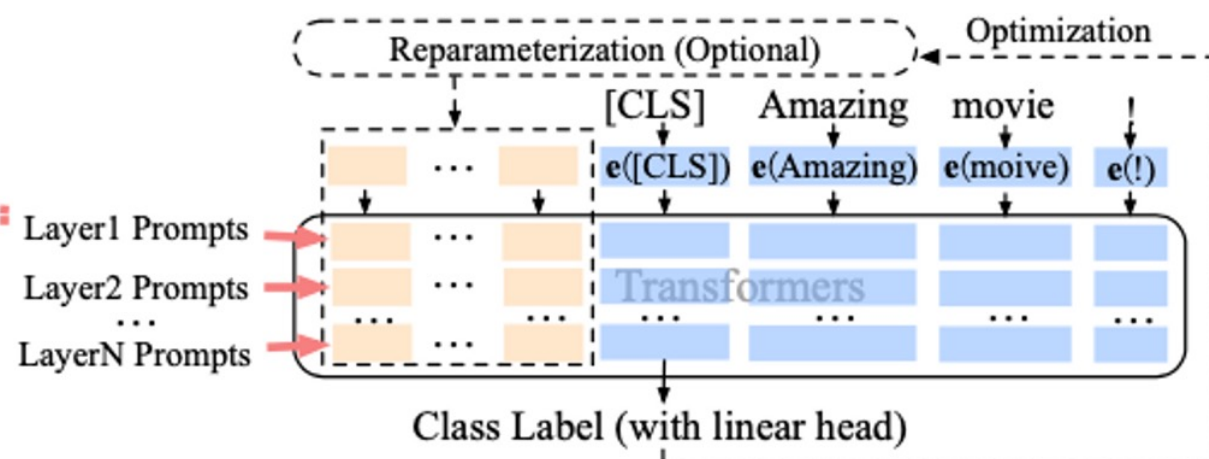
Prompt tuning



Prompt tuning VS P-tuning V2



(a) Lester et al. & P-tuning (Frozen, 10-billion-scale, simple tasks)

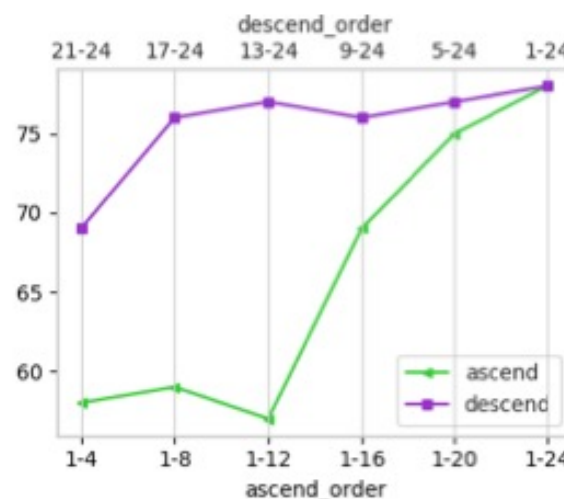


(b) P-tuning v2 (Frozen, most scales, most tasks)

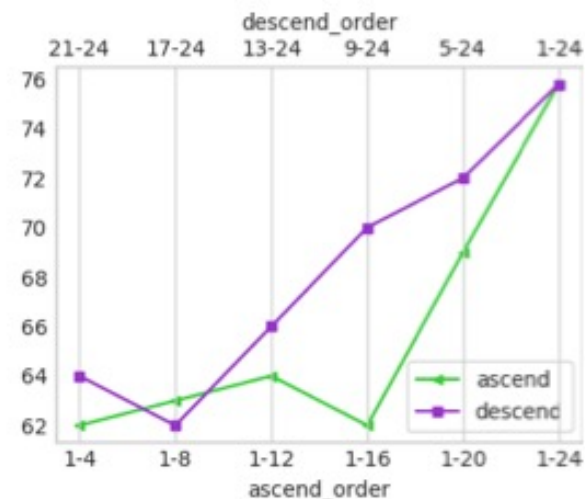
P-tuning V2

Method	Task	Re-param.	Deep PT	Multi-task	No verb.
P-tuning (Liu et al., 2021)	KP NLU	LSTM	-	-	-
PROMPTTUNING (Lester et al., 2021)	NLU	-	-	✓	-
Prefix Tuning (Li and Liang, 2021)	NLG	MLP	✓	-	-
SOFT PROMPTS (Qin and Eisner, 2021)	KP	-	✓	-	-
P-tuning v2 (Ours)	NLU SeqTag	(depends)	✓	✓	✓

Table 1: Conceptual comparison between P-tuning v2 and existing Prompt Tuning approaches (KP: Knowledge Probe; SeqTag: Sequence Tagging; Re-param.: Reparameterization; No verb.: No verbalizer).



(a) RTE



(b) BoolQ

Experiments

	#Size	BoolQ			CB			COPA			MultiRC (F1a)		
		FT	PT	PT-2	FT	PT	PT-2	FT	PT	PT-2	FT	PT	PT-2
BERT _{large}	335M	77.7	67.2	<u>75.8</u>	94.6	80.4	94.6	<u>69.0</u>	55.0	73.0	<u>70.5</u>	59.6	70.6
RoBERTa _{large}	355M	86.9	62.3	<u>84.8</u>	<u>98.2</u>	71.4	100	94.0	63.0	<u>93.0</u>	85.7	59.9	<u>82.5</u>
GLM _{xlarge}	2B	88.3	79.7	<u>87.0</u>	96.4	<u>76.4</u>	96.4	93.0	<u>92.0</u>	91.0	<u>84.1</u>	77.5	84.4
GLM _{xxlarge}	10B	<u>88.7</u>	88.8	88.8	98.7	<u>98.2</u>	96.4	98.0	98.0	98.0	88.1	<u>86.1</u>	88.1

	#Size	ReCoRD (F1)			RTE			WiC			WSC		
		FT	PT	PT-2	FT	PT	PT-2	FT	PT	PT-2	FT	PT	PT-2
BERT _{large}	335M	<u>70.6</u>	44.2	72.8	<u>70.4</u>	53.5	78.3	<u>74.9</u>	63.0	75.1	68.3	64.4	68.3
RoBERTa _{large}	355M	<u>89.0</u>	46.3	89.3	<u>86.6</u>	58.8	89.5	75.6	56.9	<u>73.4</u>	<u>63.5</u>	64.4	<u>63.5</u>
GLM _{xlarge}	2B	<u>91.8</u>	82.7	91.9	90.3	<u>85.6</u>	90.3	74.1	71.0	<u>72.0</u>	95.2	87.5	<u>92.3</u>
GLM _{xxlarge}	10B	94.4	87.8	<u>92.5</u>	93.1	<u>89.9</u>	93.1	75.7	71.8	<u>74.0</u>	95.2	<u>94.2</u>	93.3

Table 2: Results on SuperGLUE development set. P-tuning v2 surpasses P-tuning & [Lester et al. \(2021\)](#) on models smaller than 10B, matching the performance of fine-tuning across different model scales. (FT: fine-tuning; PT: [Lester et al. \(2021\)](#) & P-tuning; PT-2: P-tuning v2; **bold**: the best; underline: the second best).

Experiments

	#Size	CoNLL03				OntoNotes 5.0				CoNLL04			
		FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2
BERT _{large}	335M	92.8	81.9	90.2	<u>91.0</u>	89.2	74.6	<u>86.4</u>	86.3	<u>85.6</u>	73.6	84.5	86.6
RoBERTa _{large}	355M	<u>92.6</u>	86.1	92.8	92.8	89.8	<u>80.8</u>	89.8	89.8	<u>88.8</u>	76.2	88.4	90.6
DeBERTa _{xlarge}	750M	93.1	<u>90.2</u>	93.1	93.1	<u>90.4</u>	85.1	<u>90.4</u>	90.5	<u>89.1</u>	82.4	86.5	90.1

	#Size	SQuAD 1.1 dev (EM / F1)								SQuAD 2.0 dev (EM / F1)							
		FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2
BERT _{large}	335M	84.2	91.1	1.0	8.5	77.8	86.0	<u>82.3</u>	<u>89.6</u>	78.7	81.9	50.2	50.2	69.7	73.5	<u>72.7</u>	<u>75.9</u>
RoBERTa _{large}	355M	88.9	94.6	1.2	12.0	<u>88.5</u>	<u>94.4</u>	88.0	94.1	86.5	89.4	50.2	50.2	82.1	85.5	<u>83.4</u>	<u>86.7</u>
DeBERTa _{xlarge}	750M	<u>90.1</u>	<u>95.5</u>	2.4	19.0	90.4	95.7	89.6	95.4	<u>88.3</u>	<u>91.1</u>	50.2	50.2	88.4	91.1	88.1	90.8

	#Size	CoNLL12				CoNLL05 WSJ				CoNLL05 Brown			
		FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2	FT	PT	PT-2	MPT-2
BERT _{large}	335M	<u>84.9</u>	64.5	83.2	85.1	88.5	76.0	<u>86.3</u>	88.5	<u>82.7</u>	70.0	80.7	83.1
RoBERTa _{large}	355M	86.5	67.2	84.6	<u>86.2</u>	90.2	76.8	89.2	<u>90.0</u>	<u>85.6</u>	70.7	84.3	85.7
DeBERTa _{xlarge}	750M	<u>86.5</u>	74.1	85.7	87.1	91.2	82.3	<u>90.6</u>	91.2	<u>86.9</u>	77.7	86.3	87.0

Table 3: Results on Named Entity Recognition (NER), Question Answering (Extractive QA), and Semantic Role Labeling (SRL). All metrics in NER and SRL are micro-f1 score. (FT: fine-tuning; PT: P-tuning & [Lester et al. \(2021\)](#); PT-2: P-tuning v2; MPT-2: Multi-task P-tuning v2; **bold**: the best; underline: the second best).

main1 branch0 tags

Go to fileAdd file<> Code

	rainatam remove colbert eval script	1cd083e on Nov 4, 2022	🕒 35 commits
📁 PT-Retrieval	remove colbert eval script		5 months ago
📁 figures	initial push		2 years ago
📁 model	fix bug		2 years ago
📁 run_script	initial push		2 years ago
📁 search_script	initial push		2 years ago
📁 tasks	initial push		2 years ago
📁 training	initial push		2 years ago
📄 LICENSE	Create LICENSE		2 years ago
📄 README.md	Update README.md		6 months ago
📄 arguments.py	initial push		2 years ago
📄 requirements.txt	initial push		2 years ago
📄 run.py	initial push		2 years ago
📄 search.py	initial push		2 years ago

☰ README.md

P-tuning v2

About

An optimized deep prompt tuning strategy comparable to fine-tuning across scales and tasks

natural-language-processing

pretrained-language-modelprompt-tuning

p-tuningparameter-efficient-learning

📖 Readme

📄 Apache-2.0 license

☆ 915 stars

👁 29 watching

🍴 97 forks

Report repository

Contributors3

Xiao9905 Shaw

rainatam Tam WengLam

sofyc

Languages

Python96.7%

Shell3.3%

P-tuning v2

Source codes and data for

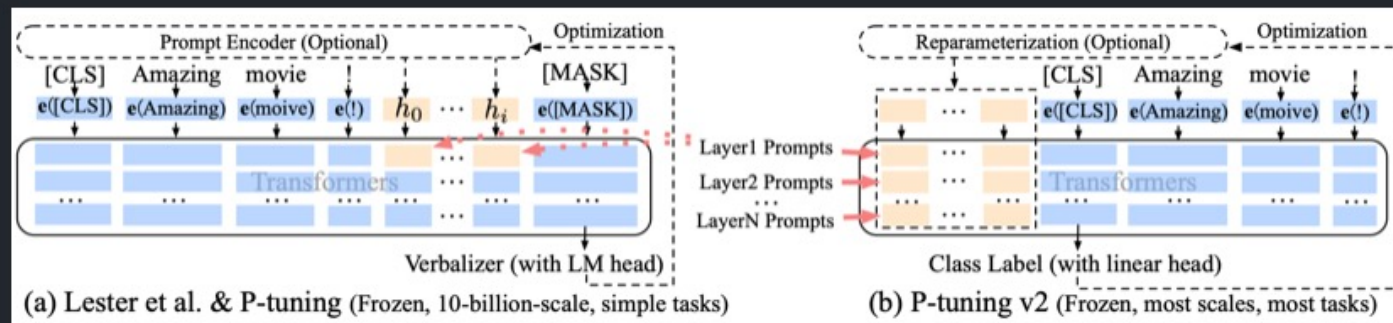
- [ACL 2022] [P-Tuning v2: Prompt Tuning Can Be Comparable to Finetuning Universally Across Scales and Tasks](#)
- [★ New: 2022.7] [Parameter-Efficient Prompt Tuning Makes Generalized and Calibrated Neural Text Retrievers](#) [Code]

An optimized prompt tuning strategy achieving comparable performance to fine-tuning on small/medium-sized models and sequence tagging challenges.

Find our previous version [P-tuning v1](#) for knowledge probing and few-shot SuperGLUE. Your kindly starring our repo can greatly encourage us to work harder :)

You may be also interested in our recent work [GLM-130B: An Open Bilingual Pre-trained Model \(2022-10-06\)](#). It is an open-sourced LLM outperforming GPT-3 175B over various benchmarks. Get model weights, do inference and P-Tuning v2 with only 4 * RTX 3090 or 8 * RTX 2080 Ti [FOR FREE!](#)

P-tuning v2 leverages **deep prompt tuning**, which is to apply continuous prompts for every layer input of the pretrained transformer. Deep prompt tuning increases the capacity of continuous prompts and closes the gap to fine-tuning across various settings, especially for small models and hard tasks.



Thanks [@rainatam](#)'s joint effort in re-organizing codes for publishing!

Languages

Python 96.7% Shell 3.3%

```

class PrefixEncoder(torch.nn.Module):
    r'''
    The torch.nn model to encode the prefix

    Input shape: (batch-size, prefix-length)

    Output shape: (batch-size, prefix-length, 2*layers*hidden)
    ...
    def __init__(self, config):
        super().__init__()
        self.prefix_projection = config.prefix_projection
        if self.prefix_projection:
            # Use a two-layer MLP to encode the prefix
            self.embedding = torch.nn.Embedding(config.pre_seq_len, config.hidden_size)
            self.trans = torch.nn.Sequential(
                torch.nn.Linear(config.hidden_size, config.prefix_hidden_size),
                torch.nn.Tanh(),
                torch.nn.Linear(config.prefix_hidden_size, config.num_hidden_layers * 2 * config.hidden_size)
            )
        else:
            self.embedding = torch.nn.Embedding(config.pre_seq_len, config.num_hidden_layers * 2 * config.hidden_size)

    def forward(self, prefix: torch.Tensor):
        if self.prefix_projection:
            prefix_tokens = self.embedding(prefix)
            past_key_values = self.trans(prefix_tokens)
        else:
            past_key_values = self.embedding(prefix)
        return past_key_values

```

https://github.com/THUDM/P-tuning-v2/blob/main/model/prefix_encoder.py

```
class BertPrefixForSequenceClassification(BertPreTrainedModel):
    def __init__(self, config):
        super().__init__(config)
        self.num_labels = config.num_labels
        self.config = config
        self.bert = BertModel(config)
        self.dropout = torch.nn.Dropout(config.hidden_dropout_prob)
        self.classifier = torch.nn.Linear(config.hidden_size, config.num_labels)

        ...

        self.prefix_tokens = torch.arange(self.pre_seq_len).long()
        self.prefix_encoder = PrefixEncoder(config)
```


...

```
def get_prompt(self, batch_size):
    prefix_tokens = self.prefix_tokens.unsqueeze(0).expand(batch_size, -1).to(self.bert.device)
    past_key_values = self.prefix_encoder(prefix_tokens)
    # bsz, seqlen, _ = past_key_values.shape
    past_key_values = past_key_values.view(
        batch_size,
        self.pre_seq_len,
        self.n_layer * 2,
        self.n_head,
        self.n_embd
    )
    past_key_values = self.dropout(past_key_values)
    past_key_values = past_key_values.permute([2, 0, 3, 1, 4]).split(2)
    return past_key_values
```

```

def forward(
    self,
    input_ids=None,
    attention_mask=None,
    token_type_ids=None,
    position_ids=None,
    head_mask=None,
    inputs_embeds=None,
    labels=None,
    output_attentions=None,
    output_hidden_states=None,
    return_dict=None,
):
    return_dict = return_dict if return_dict is not None else self.config.use_return_dict

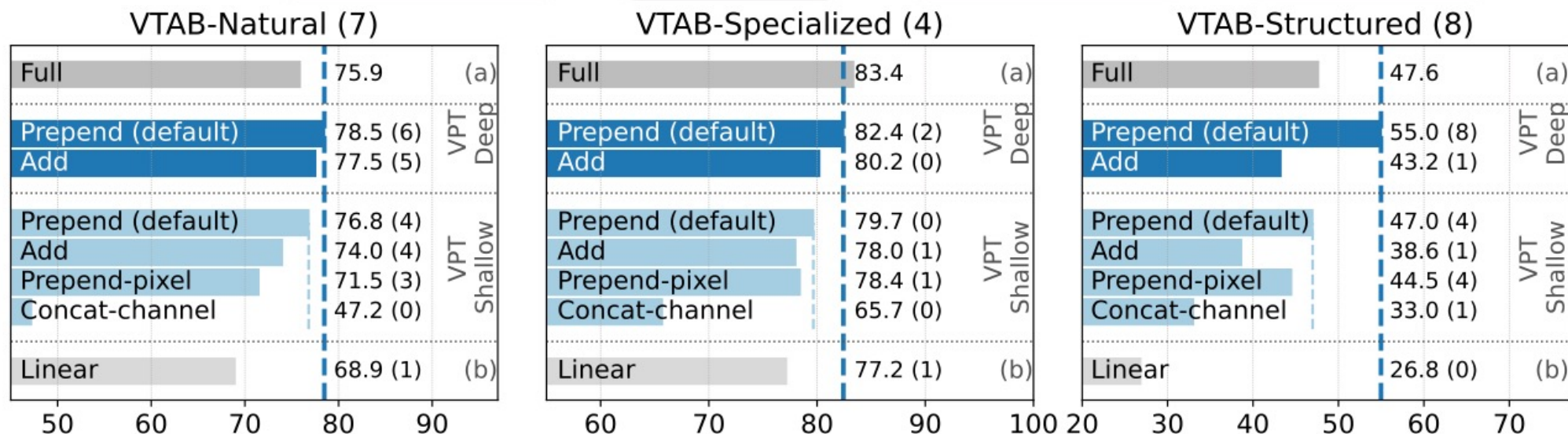
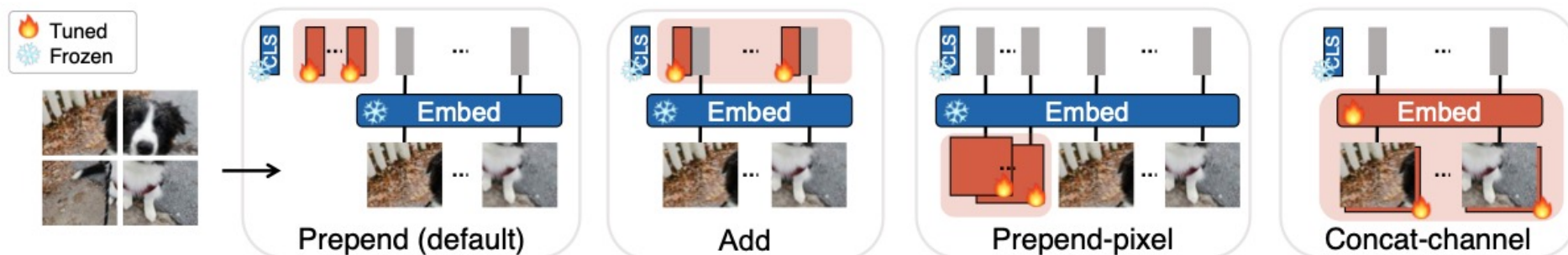
    batch_size = input_ids.shape[0]
    past_key_values = self.get_prompt(batch_size=batch_size)
    prefix_attention_mask = torch.ones(batch_size, self.pre_seq_len).to(self.bert.device)
    attention_mask = torch.cat((prefix_attention_mask, attention_mask), dim=1)

    outputs = self.bert(
        input_ids,
        attention_mask=attention_mask,
        token_type_ids=token_type_ids,
        position_ids=position_ids,

```

```
):  
    return_dict = return_dict if return_dict is not None else self.config.use_return_dict  
  
    batch_size = input_ids.shape[0]  
    past_key_values = self.get_prompt(batch_size=batch_size)  
    prefix_attention_mask = torch.ones(batch_size, self.pre_seq_len).to(self.bert.device)  
    attention_mask = torch.cat((prefix_attention_mask, attention_mask), dim=1)  
  
    outputs = self.bert(  
        input_ids,  
        attention_mask=attention_mask,  
        token_type_ids=token_type_ids,  
        position_ids=position_ids,  
        head_mask=head_mask,  
        inputs_embeds=inputs_embeds,  
        output_attentions=output_attentions,  
        output_hidden_states=output_hidden_states,  
        return_dict=return_dict,  
        past_key_values=past_key_values,  
    )  
  
    pooled_output = outputs[1]  
  
    pooled_output = self.dropout(pooled_output)  
    logits = self.classifier(pooled_output)
```

Visual prompt tuning (use case)



Q & A